

## LAB 2: Detecting Suspicious Login Activity Using Machine Learning

| Lab Overview             | Details                                                                                      |
|--------------------------|----------------------------------------------------------------------------------------------|
| Mapped Course Outcome(s) | CO3                                                                                          |
| Bloom's Level            | L3 - Apply / L4 - Analyze                                                                    |
| Total Marks              | 50                                                                                           |
| Focus Areas              | Data cleaning, Decision Tree, K-Means, confusion matrix, feature importance and false alarms |
| Platform                 | Google Colab / Python                                                                        |
| Duration                 | 4 hours                                                                                      |

### Problem Statement

Use a small numeric login-activity dataset containing failed login attempts, login hour, new-device status and downloaded data. Clean the data, train a Decision Tree to classify Normal/Suspicious activity, run K-Means clustering, interpret accuracy and confusion matrix, and test new login records.

### Lab Instructions

- Use `simple_login_activity.csv` and the starter notebook.
- Do not collect data from real accounts or networks.
- Record missing values, duplicates, training/testing rows and model results.
- Display accuracy, confusion matrix, cluster summary and feature importance.
- Test three new records and discuss one false positive and one false negative.

### Learning Outcomes

- Differentiate supervised and unsupervised learning.
- Prepare a small cyber-security dataset.
- Train and evaluate a Decision Tree.
- Use K-Means to group similar login activity.
- Interpret false positives, false negatives and model limitations.

### Q1. Dataset Inspection and Cleaning (Marks: 5)

#### AIM

To identify input columns and target label, remove duplicates and replace missing numeric values.

## Theory / Concept

- Models require complete and consistent input data.
- The median is a robust replacement for a missing numeric value.
- Duplicate records can distort analysis and evaluation.
- The target label is Normal or Suspicious; the remaining selected columns are input features.

## Algorithm / Guided Procedure

1. Load the CSV file.
2. Display the first rows and dataset information.
3. Count missing values and duplicate rows.
4. Remove duplicates.
5. Fill missing numeric values with the column median.

### PYTHON / GOOGLE COLAB SKELETON

```
import pandas as pd
import numpy as np

df = pd.read_csv('simple_login_activity.csv')

print(df.info())
print('Missing values before cleaning:')
print(df.isna().sum())
print('Duplicate rows:', df.duplicated().sum())

df = df.drop_duplicates().copy()
df['failed_logins'] = df['failed_logins'].fillna(
    df['failed_logins'].median()
)
df['download_mb'] = df['download_mb'].fillna(
    df['download_mb'].median()
)

print('Missing values after cleaning:', df.isna().sum().sum())
print('Duplicate rows after cleaning:', df.duplicated().sum())
print('Final rows:', len(df))
```

### SAMPLE / EXPECTED OUTPUT

Expected: missing values after cleaning = 0, duplicate rows after cleaning = 0, final rows approximately 100.

## Points to Record in Lab File

- Before/after cleaning summary.

- List of input columns and target label.
- Screenshot of the cleaned first five rows.

## Q2. Training and Testing Split (Marks: 5)

### AIM

To separate the features and label and create training and testing sets.

### Theory / Concept

- Supervised learning uses labelled examples.
- Training data is used to learn; testing data estimates performance on unseen records.
- A stratified split maintains a similar class proportion in both subsets.

### Algorithm / Guided Procedure

1. Select failed\_logins, login\_hour, new\_device and download\_mb.
2. Select label as the target.
3. Use 75% of rows for training and 25% for testing.
4. Set random\_state for reproducibility.

### PYTHON / GOOGLE COLAB SKELETON

```
from sklearn.model_selection import train_test_split

features = [
    'failed_logins', 'login_hour',
    'new_device', 'download_mb'
]

X = df[features]
y = df['label']

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.25,
    random_state=42,
    stratify=y
)

print('Training rows:', len(X_train))
print('Testing rows:', len(X_test))
```

### SAMPLE / EXPECTED OUTPUT

For a 100-row dataset: Training rows = 75 and Testing rows = 25.

### Points to Record in Lab File

- Feature and target definitions.
- Training/testing row counts.

- One sentence explaining why testing on training data is misleading.

### Q3. Decision Tree Training and Evaluation (Marks: 10)

#### AIM

To train a Decision Tree and interpret accuracy and the confusion matrix.

#### Theory / Concept

- A Decision Tree learns a sequence of if-then decisions from labelled data.
- Accuracy measures the overall proportion of correct predictions.
- The confusion matrix separates correct Normal/Suspicious decisions from false alarms and missed attacks.
- A shallow max\_depth improves readability and reduces overfitting.

#### Algorithm / Guided Procedure

1. Create a DecisionTreeClassifier with max\_depth=4.
2. Train it using X\_train and y\_train.
3. Predict X\_test.
4. Calculate accuracy.
5. Display a confusion matrix for Normal and Suspicious.

#### PYTHON / GOOGLE COLAB SKELETON

```
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (
    accuracy_score,
    ConfusionMatrixDisplay
)

model = DecisionTreeClassifier(
    max_depth=4,
    random_state=42
)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)

print(f'Accuracy: {accuracy * 100:.2f}%')

ConfusionMatrixDisplay.from_predictions(
    y_test,
    y_pred,
    labels=['Normal', 'Suspicious']
)
plt.title('Decision Tree Confusion Matrix')
plt.show()
```

**SAMPLE / EXPECTED OUTPUT**

Typical supplied-seed result: approximately 88% accuracy. A representative confusion matrix is  $\begin{bmatrix} 19 & 0 \\ 3 & 3 \end{bmatrix}$ .

**Points to Record in Lab File**

- Accuracy value.
- Confusion matrix screenshot.
- Two-sentence interpretation of errors.
- Reason why accuracy below 100% can be realistic.

**Q4. K-Means Clustering (Marks: 10)****AIM**

To apply an unsupervised method and compare clusters with the known labels.

**Theory / Concept**

- K-Means groups records according to numeric similarity and does not use Normal/Suspicious labels.
- Features should be standardized because they use different numeric scales.
- Clusters are not guaranteed to match security labels perfectly.
- A cluster with higher failed logins, new-device use, unusual time or larger downloads may appear more suspicious.

**Algorithm / Guided Procedure**

1. Standardize the four features.
2. Create KMeans with two clusters.
3. Assign each record a cluster number.
4. Display mean feature values per cluster.
5. Compare clusters with the known labels using a cross-tabulation.

**PYTHON / GOOGLE COLAB SKELETON**

```
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

kmeans = KMeans(
    n_clusters=2,
    random_state=42,
    n_init=10
)
```

```
df['cluster'] = kmeans.fit_predict(X_scaled)

cluster_summary = df.groupby('cluster')[features].mean().round(2)
label_summary = pd.crosstab(df['cluster'], df['label'])

display(cluster_summary)
display(label_summary)
```

### SAMPLE / EXPECTED OUTPUT

Two cluster summaries and a cross-tabulation showing how Normal and Suspicious records are distributed.

## Points to Record in Lab File

- Cluster-mean table.
- Cluster-versus-label cross-tabulation.
- Identification of the more suspicious-looking cluster with justification.

## Q5. Feature Importance and New Record Prediction (Marks: 5)

### AIM

To identify the features used most by the tree and predict three new login records.

## Theory / Concept

- Feature importance indicates how much a trained tree used each feature to split the data.
- Importance is model-specific and does not prove causation.
- New-record testing demonstrates how the model behaves on unseen inputs.

## Algorithm / Guided Procedure

1. Create a sorted feature-importance table.
2. Plot a labelled importance chart.
3. Create three new login records.
4. Predict Normal/Suspicious for each record.

### PYTHON / GOOGLE COLAB SKELETON

```
importance = pd.DataFrame({
    'feature': features,
    'importance': model.feature_importances_
}).sort_values('importance', ascending=False)

display(importance)

plt.bar(importance['feature'], importance['importance'])
plt.title('Decision Tree Feature Importance')
```

```
plt.ylabel('Importance')
plt.xticks(rotation=25)
plt.tight_layout()
plt.show()

new_records = pd.DataFrame([
    {'failed_logins': 0, 'login_hour': 10,
     'new_device': 0, 'download_mb': 45},
    {'failed_logins': 8, 'login_hour': 2,
     'new_device': 1, 'download_mb': 180},
    {'failed_logins': 2, 'login_hour': 23,
     'new_device': 1, 'download_mb': 900}
])

new_records['prediction'] = model.predict(new_records)
display(new_records)
```

### SAMPLE / EXPECTED OUTPUT

A four-bar feature-importance chart and predictions for three new records. The leading feature may vary with the generated dataset.

### Points to Record in Lab File

- Feature-importance table and chart.
- Three new-record predictions.
- One sentence for each prediction explaining the likely indicators.

## Q6. False Positives, False Negatives and Reflection (Marks: 5)

### AIM

To explain two important model errors and summarize the suitability of the model.

### Theory / Concept

- False positive: legitimate activity is incorrectly marked Suspicious.
- False negative: malicious activity is incorrectly marked Normal.
- False positives waste analyst time; false negatives can miss a real attack.
- Thresholds, data quality and human review affect operational safety.

### Algorithm / Guided Procedure

1. Write one realistic false-positive example.
2. Write one realistic false-negative example.
3. State one limitation of the dataset.
4. Recommend when a human analyst must review the result.
5. Save the completed result table.

### PYTHON / GOOGLE COLAB SKELETON

```
print(
    'False positive example:',
    'Legitimate late-night work on a new device is marked Suspicious.'
)
print(
    'False negative example:',
    'A disguised attack with few obvious indicators is marked Normal.'
)

df.to_csv('Lab2_Login_ML_Result.csv', index=False)
print('Saved: Lab2_Login_ML_Result.csv')
```

### SAMPLE / EXPECTED OUTPUT

A short error analysis and a saved CSV file.

## Points to Record in Lab File

- False-positive and false-negative examples.
- One limitation and one human-review rule.
- Lab2\_Login\_ML\_Result.csv.

## Assessment Scheme

| Question / Component | Evaluation Focus                                                  | Marks |
|----------------------|-------------------------------------------------------------------|-------|
| Q1                   | Dataset inspection and cleaning                                   | 5     |
| Q2                   | Training/testing split                                            | 5     |
| Q3                   | Decision Tree, accuracy and confusion matrix                      | 10    |
| Q4                   | K-Means clustering and interpretation                             | 10    |
| Q5                   | Feature importance and three predictions                          | 5     |
| Q6                   | Error analysis, reflection and saved result                       | 5     |
| Lab 2 Assessment     | Correctness, interpretation, code quality, demonstration and viva | 10    |
| Total                |                                                                   | 50    |

## Submission Checklist

| Required Evidence                             | Check                    |
|-----------------------------------------------|--------------------------|
| Cleaned dataset summary.                      | <input type="checkbox"/> |
| Training/testing evidence.                    | <input type="checkbox"/> |
| Decision Tree accuracy and confusion matrix.  | <input type="checkbox"/> |
| K-Means cluster summary and label comparison. | <input type="checkbox"/> |

| Required Evidence                                          | Check                    |
|------------------------------------------------------------|--------------------------|
| Feature-importance chart and three new-record predictions. | <input type="checkbox"/> |
| False-positive/false-negative discussion and saved CSV.    | <input type="checkbox"/> |